UNIVERSITE DE YAOUNDE I

Faculte des Sciences

Departement d'Informatique

Rapport d'Implementation

API de Transaction Bancaire

Devoir 304 — Conception d'API REST en JavaScript

Etudiant : AMAYEN BOUSSIOM THIERRY MARTIN

Matricule : 25I2177

Niveau : Licence 3 ICT

Etablissement : Universite de Yaounde I

Depot GitHub : https://github.com/AmayenICTL3/TPICT304

Date : Avril 2026

« Conception et implementation d'une API REST en JavaScript pour la gestion de comptes bancaires et de transactions. »

0 Table des matières

1 Introduction et contexte **3**

1.1 Contexte du projet . . . 3

1.2 Probleme resolu . . . 3

1.3 Choix technologiques . . . 3

2 Architecture du projet **3**

2.1 Structure des fichiers . . . 3

2.2 Architecture de server.js . . . 4

3 Specifications fonctionnelles **4**

3.1 Acteurs du systeme . . . 4

3.2 Les 5 fonctions du systeme . . . 4

3.3 Regles de gestion . . . 5

4 Specifications non fonctionnelles **5**

5 Implementation detaillee — server.js **5**

5.1 Initialisation et configuration . . . 5

5.2 Fonctions utilitaires . . . 6

5.2.1 Envoyer une reponse JSON . . . 6

5.2.2 Lire le corps JSON d'une requete . . . 6

5.2.3 Fonctions metier . . . 7

5.3 Le routeur HTTP . . . 8

5.4 Fonction 1 — Creer un compte (POST /accounts) . . . 8

5.5 Fonction 2 — Lister les comptes (GET /accounts) . . . 9

5.6 Fonction 3 — Consulter un compte (GET /accounts/{id}) . . . 10

5.7 Fonctions 4 et 5 — Depot et Retrait . . . 10

6 Cas de test **11**

6.1 Tableau complet des 23 cas de test . . . 11

6.2 Test manuel detaille — Scenario complet . . . 13

6.2.1 Etape 1 — Verifier la liste vide (CT-06) . . . 13

6.2.2 Etape 2 — Creer un compte (CT-01) . . . 13

6.2.3 Etape 3 — Tester les cas d'erreur (CT-03, CT-04, CT-05) . . . 13

6.2.4 Etape 4 — Effectuer un depot (CT-10) 14
6.2.5 Etape 5 — Effectuer un retrait valide (CT-15) 14
6.2.6 Etape 6 — Tester le solde insuffisant (CT-17) 14
6.2.7 Etape 7 — Consulter l'historique (CT-22) 14
6.2.8 Etape 8 — Tester un ID inexistant (CT-09) 15

7 Documentation Swagger **15**
7.1 Fonctionnement 15
7.2 Exemples interactifs dans Swagger 16
7.3 Comment utiliser Swagger pour les tests manuels 16

8 Lancement et deploiement **16**
8.1 Lancement en local 16
8.2 package.json 16

9 Conclusion **17**

1 Introduction et contexte

1.1 Contexte du projet

Ce rapport decrit de facon complete l'implementation d'une **API REST de transaction bancaire en JavaScript** realisee dans le cadre du Devoir 304 de Licence 3 ICT.

L'API permet a des clients de gerer leurs comptes bancaires : creer un compte, consulter son solde, effectuer des depots et des retraits, et visualiser l'historique de leurs operations.

1.2 Probleme resolu

Les etablissements financiers ont besoin de solutions numeriques pour automatiser la gestion des operations bancaires. Cette API constitue le back-end d'un tel systeme, consommable par une application web, mobile ou desktop.

1.3 Choix technologiques

Composant	Technologie	Raison du choix
Langage	JavaScript (Node.js)	Ubiquite, asynchrone natif, ecosystem NPM
Serveur HTTP	Module natif `http`	Aucune dependance externe, pedagogique
Documentation	Swagger UI (OpenAPI 3.0)	Standard industriel, interface interactive
Persistance	Memoire vive (tableau)	Simple, sans configuration, adapte au devoir
Identifiants	Module natif `crypto`	UUID v4 generes sans bibliotheque externe

> **Pourquoi JavaScript pur sans framework ?** L'objectif pedagogique est de comprendre le fonctionnement d'un serveur HTTP de l'interieur, sans la magie d'un framework comme Express ou Fastify. Le module `http` de Node.js est utilise directement.

2 Architecture du projet

2.1 Structure des fichiers

```
TPICT304/
|
|-- server.js              # Serveur HTTP : toute la logique de l'API
|-- openapi.json           # Specification OpenAPI 3.0 (Swagger)
|-- package.json           # Metadonnees Node.js
|-- CAHIER_DE_CHARGES.md   # Cahier des charges
|-- CAHIER_DE_CHARGES.pdf  # Version PDF du cahier des charges
|
|-- public/
    |-- swagger-ui/
        |-- swagger-ui-bundle.js  # Interface Swagger UI
```

```
        |-- swagger-ui.css
        |-- favicon.svg
```

Listing 1 – Arborescence du projet

2.2 Architecture de server.js

Contrairement a un projet Python multi-fichiers, toute la logique est contenue dans **un seul fichier** `server.js`. Il est organise en blocs fonctionnels distincts :

Bloc	Contenu	Role
1	Utilitaires	`json()`, `sendFile()`, `readJsonBody()`, `round()`
2	Logique metier	`generateAccountNumber()`, `accountSummary()`, `findAccount()`
3	Interface Swagger	`docsHtml()`
4	Routeur HTTP	`http.createServer()` avec gestion de toutes les routes

3 Specifications fonctionnelles

3.1 Acteurs du systeme

Client bancaire : l'utilisateur principal. Il peut :

- ouvrir un compte bancaire
- consulter les informations de son compte
- effectuer des depots et des retraits
- consulter l'historique de ses transactions

Administrateur systeme : supervise le systeme. Il peut :

- consulter la liste complete des comptes
- suivre les transactions

3.2 Les 5 fonctions du systeme

#	Fonction	Methode HTTP	URL
F1	Creer un compte	POST	`/accounts`
F2	Lister les comptes	GET	`/accounts`
F3	Consulter un compte	GET	`/accounts/{id}`
F4	Effectuer un depot	POST	`/accounts/{id}/deposit`
F5	Effectuer un retrait	POST	`/accounts/{id}/withdraw`
F+	Historique des transactions	GET	`/accounts/{id}/transactions`

3.3 Regles de gestion

1. Chaque compte possede un identifiant unique (UUID)
2. Chaque compte possede un numero unique au format `ACC-AAAAMMJJ-XXXXXXXX`
3. Le solde initial doit etre superieur ou egal a zero
4. Un depot doit avoir un montant strictement positif
5. Un retrait doit avoir un montant strictement positif
6. Un retrait est refuse si le solde est insuffisant
7. Chaque transaction modifie immediatement le solde
8. Chaque transaction est conservee dans l'historique du compte

4 Specifications non fonctionnelles

Critere	Exigence	Comment c'est respecte
Simplicite	Code lisible et pedagogique	Un seul fichier, fonctions courtes et bien nommees
Performance	Reponses rapides	Donnees en memoire, pas d'I/O disque
Fiabilite	Donnees correctes en execution	Tableau JavaScript synchronise, pas de race condition
Portabilite	Fonctionne sans dependances	Module `http` natif, seul `crypto` utilise
Maintenabilite	Code structure clairement	Blocs distincts : utilitaires, metier, routeur
Disponibilite	API accessible	Fonctionne tant que le processus Node.js est actif

5 Implementation detaillee — server.js

5.1 Initialisation et configuration

```
const http = require("http");        // serveur HTTP natif
const fs = require("fs");            // lecture de fichiers (Swagger)
const path = require("path");        // gestion des chemins
const { randomUUID } = require("crypto");  // generation UUID v4

const HOST = "0.0.0.0";             // ecoute sur toutes les interfaces
const PORT = Number(process.env.PORT || process.argv[2] || 8003);
// PORT : variable d'environnement > argument CLI > defaut 8003

const accounts = [];    // tableau en memoire : stocke tous les comptes
```

Listing 2 – server.js — Imports et configuration

Persistance en memoire : Les donnees sont stockees dans le tableau `accounts` pendant l'execution du serveur. Elles sont perdues a l'arret. C'est un choix delibere pour ce devoir academique (pas de base de donnees requise).

5.2 Fonctions utilitaires

5.2.1 Envoyer une reponse JSON

```javascript
function json(response, statusCode, payload) {
  const body = JSON.stringify(payload);   // serialise l'objet en chaine JSON
  response.writeHead(statusCode, {
    "Content-Type": "application/json; charset=utf-8",
    "Content-Length": Buffer.byteLength(body),  // taille en octets
  });
  response.end(body);  // envoie la reponse et ferme la connexion
}
```

Listing 3 – server.js — Fonction json()

Cette fonction est appelee partout dans le routeur pour retourner des reponses JSON avec le bon code HTTP et les bons en-tetes.

5.2.2 Lire le corps JSON d'une requete

```javascript
function readJsonBody(request) {
  return new Promise((resolve, reject) => {
    let raw = "";

    // Node.js recoit le corps en morceaux (chunks) de facon asynchrone
    request.on("data", (chunk) => {
      raw += chunk;   // concatene les morceaux
    });

    request.on("end", () => {
      if (!raw) {
        resolve({});   // corps vide -> objet vide
        return;
      }
      try {
        resolve(JSON.parse(raw));   // parse le JSON
      } catch (error) {
        reject(error);   // JSON invalide -> erreur
      }
    });

    request.on("error", reject);
  });
}
```

Listing 4 – server.js — Fonction readJsonBody()

> **Pourquoi une Promise ?** HTTP en Node.js est asynchrone par nature : le corps d'une requete arrive en morceaux successifs. La Promise permet d'attendre que tous les morceaux soient recus avant de parser le JSON, en utilisant `await`.

5.2.3 Fonctions metier

```javascript
function round(value) {
  // Arrondit a 2 decimales pour eviter les erreurs flottantes
  // ex: 0.1 + 0.2 = 0.30000000000000004 en JS
  return Math.round(Number(value) * 100) / 100;
}

function generateAccountNumber() {
  // Genere: ACC-20260422-A1B2C3D4
  const datePart = new Date().toISOString().slice(0,
      10).replace(/-/g, "");
  const randomPart = randomUUID().replace(/-/g, "").slice(0,
      8).toUpperCase();
  return `ACC-${datePart}-${randomPart}`;
}

function accountSummary(account) {
  // Retourne le compte SANS les transactions (pour GET /accounts)
  return {
    id: account.id,
    account_number: account.account_number,
    full_name: account.full_name,
    phone_number: account.phone_number,
    email: account.email,
    balance: account.balance,
    created_at: account.created_at,
  };
}

function accountDetails(account) {
  // Retourne le compte AVEC les transactions (pour GET
      /accounts/:id)
  return {
    ...accountSummary(account),   // spread: copie tous les champs
        du resume
    transactions: account.transactions,
  };
}

function findAccount(accountId) {
  // Cherche un compte par son UUID dans le tableau en memoire
  return accounts.find((account) => account.id === accountId);
  // Retourne undefined si non trouve
}
```

Listing 5 – server.js — Fonctions metier

5.3 Le routeur HTTP

Le serveur utilise un unique gestionnaire de requetes qui analyse la methode HTTP et le chemin URL pour dispatcher vers la bonne logique.

```javascript
const server = http.createServer(async (request, response) => {
  const url = new URL(request.url, `http://${HOST}:${PORT}`);
  const pathname = url.pathname;
  // request.method = "GET", "POST", etc.
  // pathname = "/accounts", "/accounts/uuid", etc.

  // === Routes statiques ===
  if (request.method === "GET" && pathname === "/") { ... }
  if (request.method === "GET" && pathname === "/docs") { ... }
  if (request.method === "GET" && pathname === "/openapi.json") { ... }
  if (request.method === "GET" && pathname.startsWith("/public/")) { ... }

  // === Routes dynamiques /accounts ===
  if (pathname === "/accounts" && request.method === "GET") { ... }
  if (pathname === "/accounts" && request.method === "POST") { ... }

  // === Routes avec parametre /accounts/:id ===
  const accountMatch = pathname.match(/^\/accounts\/([^/]+)$/);
  // accountMatch[1] = l'UUID extrait de l'URL

  // === Routes avec action /accounts/:id/(deposit|withdraw|transactions) ===
  const txMatch = pathname.match(/^\/accounts\/([^/]+)\/(deposit|withdraw|transactions)$/);
  // txMatch[1] = UUID, txMatch[2] = "deposit" ou "withdraw" ou "transactions"
});
```

Listing 6 – server.js — Squelette du routeur

> **Expressions regulieres pour le routage :** En l'absence de framework, les routes dynamiques sont identifiees par des expressions regulieres. Le groupe de capture `([^/]+)` extrait l'UUID, et `(deposit|withdraw|transactions)` identifie l'action.

5.4 Fonction 1 — Creer un compte (POST /accounts)

```javascript
if (pathname === "/accounts" && request.method === "POST") {
  try {
    const payload = await readJsonBody(request);  // lit le corps JSON

    // Validation 1 : champs obligatoires
    if (!payload.full_name || !payload.phone_number) {
      json(response, 400, { detail: "full_name et phone_number sont obligatoires." });
      return;
    }

```

```javascript
    // Validation 2 : solde initial non negatif
    const initialBalance = round(payload.initial_balance || 0);
    if (initialBalance < 0) {
      json(response, 400, { detail: "Le solde initial doit etre positif ou nul." });
      return;
    }

    // Construction de l'objet compte
    const account = {
      id: randomUUID(),                      // UUID v4 unique
      account_number: generateAccountNumber(),
      full_name: payload.full_name,
      phone_number: payload.phone_number,
      email: payload.email || null,          // null si absent
      balance: initialBalance,
      created_at: new Date().toISOString(),
      transactions: [],                      // tableau vide au depart
    };

    // Si solde initial > 0, on cree une transaction "depot initial"
    if (initialBalance > 0) {
      account.transactions.push({
        transaction_id: randomUUID(),
        transaction_type: "deposit",
        amount: initialBalance,
        description: "Solde initial",
        balance_after: initialBalance,
        created_at: new Date().toISOString(),
      });
    }

    accounts.push(account);                  // ajoute au tableau memoire
    json(response, 201, accountSummary(account));  // HTTP 201 Created
  } catch {
    json(response, 400, { detail: "Corps JSON invalide." });
  }
  return;
}
```

Listing 7 – server.js — POST /accounts

Etapes de traitement :

1. Lecture asynchrone du corps JSON via `readJsonBody`
2. Validation des champs obligatoires (`full_name`, `phone_number`)
3. Validation du solde initial (≥ 0)
4. Construction de l'objet compte avec UUID et numero generes
5. Si solde > 0 : ajout d'une transaction initiale dans l'historique
6. Ajout dans le tableau `accounts`
7. Reponse HTTP 201 avec le resume du compte (sans les transactions)

5.5 Fonction 2 — Lister les comptes (GET /accounts)

```javascript
if (pathname === "/accounts" && request.method === "GET") {
  // accounts.map(accountSummary) : transforme chaque compte
  // en version "sans transactions" pour alleger la reponse
  json(response, 200, accounts.map(accountSummary));
  return;
}
```

Listing 8 – server.js — GET /accounts

Simple mais efficace : `Array.map()` applique `accountSummary` a chaque element du tableau et retourne un nouveau tableau avec uniquement les champs pertinents.

5.6 Fonction 3 — Consulter un compte (GET /accounts/{id})

```javascript
const accountMatch = pathname.match(/^\/accounts\/([^/]+)$/);
if (accountMatch && request.method === "GET") {
  const account = findAccount(accountMatch[1]);  // recherche par
      UUID
  if (!account) {
    json(response, 404, { detail: "Compte introuvable." });
    return;
  }
  json(response, 200, accountDetails(account));  // avec transactions
  return;
}
```

Listing 9 – server.js — GET /accounts/{account_id}

`accountMatch[1]` contient l'UUID extrait de l'URL par l'expression reguliere. Si `findAccount` retourne `undefined`, on repond HTTP 404.

5.7 Fonctions 4 et 5 — Depot et Retrait

```javascript
const txMatch = pathname.match(
  /^\/accounts\/([^/]+)\/(deposit|withdraw|transactions)$/
);
if (txMatch) {
  const account = findAccount(txMatch[1]);   // recherche le compte
  if (!account) {
    json(response, 404, { detail: "Compte introuvable." });
    return;
  }

  const action = txMatch[2];  // "deposit", "withdraw" ou
      "transactions"

  if ((action === "deposit" || action === "withdraw") &&
      request.method === "POST") {
    try {
      const payload = await readJsonBody(request);
      const amount = round(payload.amount);

      // Validation : montant strictement positif
      if (!Number.isFinite(amount) || amount <= 0) {
```

```javascript
        json(response, 400, { detail: "Le montant doit etre strictement positif." });
        return;
      }

      // Validation specifique au retrait : solde suffisant
      if (action === "withdraw" && amount > account.balance) {
        json(response, 400, { detail: "Solde insuffisant pour effectuer ce retrait." });
        return;
      }

      // Mise a jour du solde
      account.balance = action === "deposit"
        ? round(account.balance + amount)   // depot : on ajoute
        : round(account.balance - amount);  // retrait : on soustrait

      // Enregistrement de la transaction dans l'historique
      account.transactions.push({
        transaction_id: randomUUID(),
        transaction_type: action,
        amount,
        description: payload.description || null,
        balance_after: account.balance,
        created_at: new Date().toISOString(),
      });

      json(response, 200, accountDetails(account));  // compte mis a jour
    } catch {
      json(response, 400, { detail: "Corps JSON invalide." });
    }
    return;
  }
}
```

Listing 10 – server.js — POST /accounts/{id}/(deposit|withdraw)

Modification directe de l'objet en memoire : Comme `account` est une reference vers l'objet dans le tableau `accounts`, modifier `account.balance` et `account.transactions` modifie directement la donnee en memoire. Pas besoin de "sauvegarder" explicitement.

6 Cas de test

Categories de tests logiciels

Conception et gestion des cas de test

Format de cas de test (modele 1)

Fonction 1 : Creer un compte — `POST /accounts`

Test Case ID	Test Case Description	Test Input (Test Case Value)	Expected Results	Test Case Status
CT-01	Creation valide avec tous les champs	`full_name: "AMAYEN BOUSSIOM THIERRY MARTIN"` `phone_number: "699001122"` `email: "thierry@example.com"` `initial_balance: 10000`	HTTP 201 — Compte cree avec `id` UUID et `account_number` generes. `balance = 10000`	PASS
CT-02	Creation sans email ni solde initial	`full_name: "NGUEMA CAROLE"` `phone_number: "677445566"`	HTTP 201 — `balance = 0`, `email = null`	PASS
CT-03	`full_name` absent	`phone_number: "699001122"` `initial_balance: 5000`	HTTP 400 — Erreur : champ obligatoire manquant	PASS
CT-04	`phone_number` absent	`full_name: "AMAYEN BOUSSIOM THIERRY MARTIN"` `initial_balance: 5000`	HTTP 400 — Erreur : champ obligatoire manquant	PASS
CT-05	Solde initial negatif	`full_name: "TEST"` `phone_number: "600000001"` `initial_balance: -500`	HTTP 400 — *"Le solde initial doit etre positif ou nul."*	PASS

Fonction 2 : Lister les comptes — `GET /accounts`

Test Case ID	Test Case Description	Test Input (Test Case Value)	Expected Results	Test Case Status
CT-06	Liste vide au demarrage	Aucun compte cree. `GET /accounts`	HTTP 200 — Corps : `[]`	PASS
CT-07	Liste avec comptes existants	Au moins un compte cree. `GET /accounts`	HTTP 200 — Tableau de comptes (sans transactions)	PASS

Fonction 3 : Consulter un compte — `GET /accounts/{account_id}`

Test Case ID	Test Case Description	Test Input (Test Case Value)	Expected Results	Test Case Status

CT-08	Consulter un compte existant	`account_id` = UUID valide recupere apres creation	HTTP 200 — Details du compte avec l'historique complet des transactions	PASS
CT-09	Consulter un compte inexistant	`account_id` = `00000000-0000-0000-0000-0`	HTTP 404 — *"Compte introuvable."*	PASS

Fonction 4 : Effectuer un depot — `POST /accounts/{account_id}/deposit`

Test Case ID	Test Case Description	Test Input (Test Case Value)	Expected Results	Test Case Status
CT-10	Depot valide sans description	Compte `balance = 10000` `amount: 5000`	HTTP 200 — `balance = 15000`, transaction `deposit` enregistree	PASS
CT-11	Depot valide avec description	Compte existant `amount: 3000` `description: "Depot agence"`	HTTP 200 — Libelle visible dans l'historique	PASS
CT-12	Montant egal a zero	Compte existant `amount: 0`	HTTP 400 — *"Le montant doit etre strictement positif."*	PASS
CT-13	Montant negatif	Compte existant `amount: -1000`	HTTP 400 — *"Le montant doit etre strictement positif."*	PASS
CT-14	Depot sur compte inexistant	`account_id` invalide `amount: 5000`	HTTP 404 — *"Compte introuvable."*	PASS

Fonction 5 : Effectuer un retrait — `POST /accounts/{account_id}/withdraw`

Test Case ID	Test Case Description	Test Input (Test Case Value)	Expected Results	Test Case Status
CT-15	Retrait valide, solde suffisant	Compte `balance = 10000` `amount: 2000` `description: "Retrait client"`	HTTP 200 — `balance = 8000`, transaction `withdraw` enregistree	PASS
CT-16	Retrait egal au solde disponible	Compte `balance = 10000` `amount: 10000`	HTTP 200 — `balance = 0`, compte vide	PASS
CT-17	Solde insuffisant	Compte `balance = 1000` `amount: 50000`	HTTP 400 — *"Solde insuffisant pour effectuer ce retrait."*	PASS
CT-18	Montant egal a zero	Compte existant `amount: 0`	HTTP 400 — *"Le montant doit etre strictement positif."*	PASS

CT-19	Montant negatif	Compte existant `amount: -500`	HTTP 400 — *"Le montant doit etre strictement positif."*	PASS
CT-20	Retrait sur compte inexistant	`account_id` invalide `amount: 1000`	HTTP 404 — *"Compte introuvable."*	PASS

Fonction 6 : Virement entre comptes — `POST /accounts/{account_id}/transfer`

Test Case ID	Test Case Description	Test Input (Test Case Value)	Expected Results	Test Case Status
CT-24	Virement valide entre deux comptes	Compte A `balance = 10000`, Compte B `balance = 5000` `to_account_id` = ID de B `amount: 3000`	HTTP 200 — A : `balance = 7000`, B : `balance = 8000`. Transactions `transfer_out` et `transfer_in` enregistrees	PASS
CT-25	Virement valide avec description	Deux comptes existants `amount: 1000` `description: "Remboursement loyer"`	HTTP 200 — Description visible dans les deux historiques	PASS
CT-26	Solde insuffisant pour le virement	Compte A `balance = 1000` `amount: 50000`	HTTP 400 — *"Solde insuffisant pour effectuer ce virement."*	PASS
CT-27	Virement vers le meme compte	`to_account_id` = meme `account_id` `amount: 1000`	HTTP 400 — *"Le compte source et le compte destinataire doivent etre differents."*	PASS
CT-28	Compte source inexistant	`account_id` invalide `to_account_id` valide `amount: 1000`	HTTP 404 — *"Compte introuvable."*	PASS
CT-29	Compte destinataire inexistant	`account_id` valide `to_account_id` = `00000000-0000-0000-0000-0` `amount: 1000`	HTTP 404 — *"Compte destinataire introuvable."*	PASS

Fonction + : Historique des transactions — `GET /accounts/{account_id}/transactions`

Test Case ID	Test Case Description	Test Input (Test Case Value)	Expected Results	Test Case Status

CT-21	Historique d'un compte sans operation	Compte cree avec `initial_balance = 0` `GET /accounts/{id}/transactions`	HTTP 200 — Corps : `[]`	PASS
CT-22	Historique avec plusieurs operations	Compte avec depot, retrait et virement effectues	HTTP 200 — Liste chronologique avec type, montant et solde apres	PASS
CT-23	Historique d'un compte inexistant	`account_id` invalide	HTTP 404 — *"Compte introuvable."*	PASS

6.1 Test manuel detaille — Scenario complet

Les tests manuels se font dans **Swagger UI** a l'adresse `http://localhost:8003/docs`.

6.1.1 Etape 1 — Verifier la liste vide (CT-06)

```
GET http://localhost:8003/accounts
```

Listing 11 – Requete GET /accounts

Resultat attendu : HTTP 200 avec le corps `[]`
Confirme que le serveur repond et que le tableau est vide au demarrage.

6.1.2 Etape 2 — Creer un compte (CT-01)

Dans Swagger, ouvrir **POST /accounts**, cliquer **Try it out**, selectionner l'exemple **CT-01** dans le menu deroulant *Examples*.

```
{
  "full_name": "AMAYEN BOUSSIOM THIERRY MARTIN",
  "phone_number": "699001122",
  "email": "thierry@example.com",
  "initial_balance": 10000
}
```

Listing 12 – Corps de la requete

```
{
  "id": "a3f1b2c4-d5e6-7890-abcd-ef1234567890",
  "account_number": "ACC-20260422-A1B2C3D4",
  "full_name": "AMAYEN BOUSSIOM THIERRY MARTIN",
  "phone_number": "699001122",
  "email": "thierry@example.com",
  "balance": 10000,
  "created_at": "2026-04-22T10:00:00.000Z"
}
```

Listing 13 – Reponse attendue — HTTP 201

A noter : Copier l'`id` retourne pour les etapes suivantes.

6.1.3 Etape 3 — Tester les cas d'erreur (CT-03, CT-04, CT-05)

Selectionner l'exemple **CT-05** dans le menu Examples :

```
{
  "full_name": "TEST NEGATIF",
  "phone_number": "600000001",
  "initial_balance": -500
}
```

Listing 14 – Test CT-05 : solde negatif

Resultat attendu : HTTP 400
Corps : `{"detail": "Le solde initial doit etre positif ou nul."}`

6.1.4 Etape 4 — Effectuer un depot (CT-10)

```
POST http://localhost:8003/accounts/a3f1b2c4-.../deposit
```

Listing 15 – Requete POST /accounts/{id}/deposit

Selectionner l'exemple **CT-10** :

```
{ "amount": 5000 }
```

Listing 16 – Corps

Resultat attendu : HTTP 200, balance = 15000 (10000 + 5000).
2 transactions visibles : solde initial + depot.

6.1.5 Etape 5 — Effectuer un retrait valide (CT-15)

Selectionner l'exemple **CT-15** :

```
{ "amount": 2000, "description": "Retrait client" }
```

Listing 17 – Corps

Resultat attendu : HTTP 200, balance = 13000 (15000 - 2000).

6.1.6 Etape 6 — Tester le solde insuffisant (CT-17)

Selectionner l'exemple **CT-17** :

```
{ "amount": 50000, "description": "Test solde insuffisant" }
```

Listing 18 – Corps

Resultat attendu : HTTP 400
Corps : `{"detail": "Solde insuffisant pour effectuer ce retrait."}`

6.1.7 Etape 7 — Consulter l'historique (CT-22)

```
GET http://localhost:8003/accounts/a3f1b2c4-.../transactions
```

Listing 19 – Requete GET /accounts/{id}/transactions

Resultat attendu : HTTP 200 avec 3 transactions dans l'ordre chronologique :
1. Depot initial (10 000) — balance 10 000
2. Depot (5 000) — balance 15 000
3. Retrait (2 000) — balance 13 000

6.1.8 Etape 8 — Tester un ID inexistant (CT-09)

```
GET http://localhost:8003/accounts
    /00000000-0000-0000-0000-000000000000
```

Listing 20 – Test CT-09

Resultat attendu : HTTP 404
Corps : `{"detail": "Compte introuvable."}`

7 Documentation Swagger

7.1 Fonctionnement

L'interface Swagger UI est accessible a `http://localhost:8003/docs`. Elle est servie par `server.js` depuis le dossier `public/swagger-ui/` et se nourrit du fichier `openapi.json` pour afficher la documentation interactive.

```
function docsHtml() {
  return `<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="/public/swagger-ui/swagger-ui.css">
  <title>API JavaScript - Swagger UI</title>
</head>
<body>
  <div id="swagger-ui"></div>
  <script src="/public/swagger-ui/swagger-ui-bundle.js"></script>
  <script>
    SwaggerUIBundle({
      url: '/openapi.json',          // charge le fichier de spec
      dom_id: '#swagger-ui',
      deepLinking: true,
      docExpansion: 'list',
```

```
      displayRequestDuration: true,  // affiche le temps de reponse
      filter: true,
    });
  </script>
</body>
</html>`;
}
```

Listing 21 – server.js — Generation de la page Swagger

7.2 Exemples interactifs dans Swagger

Le fichier `openapi.json` contient des exemples nommes pour chaque endpoint :

Endpoint	Exemples disponibles dans Swagger
`POST /accounts`	CT-01, CT-02, CT-03, CT-04, CT-05
`POST /{id}/deposit`	CT-10, CT-11, CT-12, CT-13
`POST /{id}/withdraw`	CT-15, CT-16, CT-17, CT-18, CT-19

7.3 Comment utiliser Swagger pour les tests manuels

1. Lancer le serveur : `node server.js`
2. Ouvrir `http://localhost:8003/docs` dans le navigateur
3. Cliquer sur un endpoint (ex : *POST /accounts*)
4. Cliquer sur **Try it out**
5. Selectionner un cas de test dans le menu deroulant **Examples**
6. Cliquer sur **Execute**
7. Verifier le code HTTP et le corps de la reponse

8 Lancement et deploiement

8.1 Lancement en local

```
# Lancer le serveur sur le port par defaut (8003)
node server.js

# Ou sur un port personnalise
node server.js 4000

# L'API est disponible sur :
#   http://localhost:8003/         (point d'entree)
#   http://localhost:8003/docs     (Swagger UI)
```

Listing 22 – Lancement du serveur

8.2 package.json

```
{
  "name": "api-bancaire-javascript",
  "version": "1.0.0",
  "description": "API REST bancaire en JavaScript pur",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  }
}
```

Listing 23 – package.json

Aucune dependance NPM : `npm install` n'est pas necessaire.

9 Conclusion

Ce projet implemente une **API REST complete en JavaScript pur** pour la gestion de comptes bancaires, sans aucune dependance externe, respectant toutes les exigences fonctionnelles et non fonctionnelles du cahier des charges.

9.0 Points cles de l'implementation

- **JavaScript pur :** module `http` natif, aucune dependance NPM externe
- **Routage par regex :** expressions regulieres pour extraire les parametres d'URL
- **Asynchronisme :** `async/await` et Promises pour la lecture du corps HTTP
- **Validation manuelle :** toutes les regles de gestion implementees explicitement
- **Documentation interactive :** 23 cas de test executables dans Swagger UI
- **Codes HTTP corrects :** 201 (creation), 200 (succes), 400 (erreur metier), 404 (non trouve)

9.0 Limites actuelles

- Pas de persistance : donnees perdues a l'arret du serveur
- Pas d'authentification
- Pas de virement entre comptes
- Pas de separation des roles client/administrateur

Ces elements representent des evolutions naturelles pour une version future.

AMAYEN BOUSSIOM THIERRY MARTIN — Matricule 25I2177 — Licence 3 ICT
Universite de Yaounde I — Avril 2026